CS546: Computer System Security Instructor: Aarushi Goel

Homework 3

Due: November 15; 2026 (11:59 PM ET)

Instructions

1. You can collaborate with one other classmate, but the final write-up should be your own. Acknowledge the person you've worked with. All submissions must be made via Gradescope.

2. You may refer to lecture slides, course materials, and references linked on the course website. You may also consult online resources, including documentation for Python, HTML, JavaScript, and SQL, to understand relevant concepts and syntax. However, **you must not** search for, copy, or use AI tools (such as ChatGPT, Claude, or Gemini) to generate solutions to any of the problems in this homework.

3. Clearly explain your reasoning for each problem. For questions involving attacks, provide the requested attack inputs or strategies and explain why they succeed. For mathematical questions, show the intermediate steps and justify your answers.

1 Cross-Site Scripting (XSS) Attacks

Consider a web application that displays user-controlled data in three different ways.

Application A:

```
<div id="welcome">
  Welcome, USER_INPUT!
</div>
```

Application B:

```
<input type="text" value="USER_INPUT">
```

Application C:

```
<script>
  const username = 'USER_INPUT';
  document.getElementById("welcome").textContent = username;
</script>
```

In each application, `USER_INPUT` is replaced directly with attacker-controlled input, without escaping or sanitization. Assume that the resulting HTML is included in a page loaded by the victim's browser, that an element with `id="welcome"` exists for Application C, and that no Content Security Policy (CSP) restricts script execution.

(30 points) For each application, construct an input that causes JavaScript code to execute when a victim loads the page. Your payload should execute `console.log("XSS")`. For each application, also explain why your attack works.

2 Vulnerable TLS Implementation

A TLS certificate contains information about a server's identity, its public key, and the Certificate Authority (CA) that issued the certificate. One important field is the *Subject Alternative Name (SAN)* extension, which lists the domain names that the certificate is valid for. For example, a simplified TLS certificate might contain:

```
Certificate:
    Subject: CN = example.com
    Issuer: Trusted CA
    Public Key: <server's public key>

    Subject Alternative Name (SAN):
        DNS: example.com
        DNS: www.example.com
        DNS: api.example.com

    Signature: <CA's digital signature>
```

This certificate can authenticate `example.com`, `www.example.com`, and `api.example.com`, but not `mail.example.com`. During a TLS connection, the client must verify that the server's certificate is valid for the hostname the client intended to contact and chains to a trusted CA. Consider the following Python implementation of a TLS client. The client establishes a TLS connection to a server and sends an HTTP request.

```python
import socket
import ssl

def secure_request(host, port=443):

    # Create a TLS context using trusted CAs.
    context = ssl.create_default_context()

    # Require TLS version 1.2 or higher.
    context.minimum_version = ssl.TLSVersion.TLSv1_2

    # Require a certificate that chains to a trusted CA.
    context.verify_mode = ssl.CERT_REQUIRED
```

```
    # The developer implements hostname verification
    # manually instead of using the built-in check.
    context.check_hostname = False

    # Establish a TCP connection.
    with socket.create_connection((host, port)) as sock:

        # Perform a TLS handshake over the TCP connection.
        # server_hostname is used for Server Name
        # Indication (SNI).
        with context.wrap_socket(
            sock, server_hostname=host
        ) as tls:

            # Retrieve the server's certificate.
            cert = tls.getpeercert()

            # Extract DNS names from the certificate's
            # Subject Alternative Name (SAN) extension.
            valid_names = [
                name.lower()
                for typ, name in cert.get("subjectAltName", [])
                if typ == "DNS"
            ]

            # Check whether the requested hostname
            # matches a DNS name in the certificate.
            if not any(
                host.lower().endswith(name)
                for name in valid_names
            ):
                raise ssl.SSLError(
                    "Certificate hostname mismatch"
                )

            # Construct an HTTP GET request.
            request = (
                f"GET /account HTTP/1.1\r\n"
                f"Host: {host}\r\n"
                f"Connection: close\r\n\r\n"
            )

            # Send the request over the TLS connection.
            tls.sendall(request.encode())

            # Read the server's response.
            return tls.recv(4096)


# Connect to the server.
secure_request("api.example.com")
```

1. **(10 points) Identify a security vulnerability in the implementation's hostname verification procedure. Explain why the procedure does not correctly authenticate the intended server.**

2. **(10 points) Suppose an attacker controls the domain `ample.com` and possesses a valid TLS certificate for this domain issued by a trusted CA. The attacker can also redirect the client's network connection intended for `api.example.com` to this attacker-controlled server.**

   Can the attacker successfully impersonate `api.example.com`? If so, describe the attack step by step, explaining why the TLS handshake succeeds and why the client accepts the attacker's certificate.

3 SQL Injection Attacks

A website authenticates users using the following SQL query:

```
query = f"""
SELECT username
FROM users
WHERE username = '{username}'
  AND password_hash = '{password_hash}'
  AND active = TRUE;
"""
```

Assume the database contains three users: `alice`, `admin`, and `suspended`. The first two accounts are active, while the third is inactive. An attacker controls the `username` field and can supply an arbitrary incorrect value for `password_hash`.

1. **(15 points) Construct an input for `username` that causes the query to return the `admin` account without knowing its password. Your attack must satisfy the following restrictions:**

   - **No SQL comments (`--` or `/* ... */`).**
   - **No semicolons or additional SQL statements.**
   - **Only the `username` field may contain the injected SQL.**

   Explain why the attack succeeds.

2. **(10 points) Can the attacker similarly force the query to return the inactive `suspended` account? Construct an input and explain why it works or explain why this is impossible.**

4 Blockchains

Consider a proof-of-work blockchain, such as Bitcoin, in which miners compete to extend the blockchain by solving computational puzzles. When multiple competing chains exist, miners follow the *longest-chain rule*, preferring the valid chain containing the most accumulated proof-of-work.

Adversarial Miners. Suppose a coalition of adversarial miners controls a fraction α, where $0 < \alpha < 1/2$, of the total mining power. The remaining miners, controlling fraction $1-\alpha$, follow the protocol honestly. The adversarial coalition wants to censor transactions belonging to a particular blacklist $\mathcal{B}$. To achieve this, the coalition adopts a strategy known as *feather forking*.

Attack Strategy. Suppose an honest miner publishes a block containing a transaction from $\mathcal{B}$. Rather than extending this block, the adversarial coalition attempts to create an alternative blockchain that excludes the blacklisted transaction. The attack proceeds as follows:

1. *Initiating the attack:* As soon as an honest miner publishes a block containing a blacklisted transaction, the adversarial miners begin mining an alternative branch starting from the parent of that block. Initially, their branch is one block behind the honest chain.

2. *Continuing the attack:* The adversarial miners exclusively extend their alternative branch, while honest miners continue extending the chain containing the blacklisted transaction.

3. *Success condition:* If the adversarial branch becomes strictly longer than the honest chain, the attack succeeds. Under the longest-chain rule, the adversarial branch is adopted, and the block containing the blacklisted transaction is discarded.

4. *Failure condition:* If the adversarial branch falls three blocks behind the honest chain, the adversarial miners abandon their fork and resume mining on the honest chain. The attack then fails.

Simplifying Assumptions. For simplicity, we can assume that each newly discovered block is mined by the adversarial coalition with probability α and by honest miners with probability $1-\alpha$, independently of previous discoveries. Also, honest miners never switch to the adversarial branch while it is shorter than or tied with the honest chain.

(15 points) Let p_0, p_1, and p_2 denote the probabilities that the adversarial coalition eventually succeeds, starting from a state in which its branch is respectively tied with, one block behind, or two blocks behind the honest chain. Derive a system of three recursive equations relating p_0, p_1, and p_2 in terms of α. Using these equations, determine the probability that the adversarial coalition successfully censors the blacklisted transaction, expressed as a function of α.